S905X4 Secure Key Provision User Guide

Revision 3.3

Amlogic, Inc.
2518 Mission College Blvd, Suite 120
Santa Clara, CA 95054
U.S.A.
www.amlogic.com

Table of Content

Revision History

Revision	Revised Date	By	Changes
0.1	Feb. 25, 2017	Peifu Jiang	Initial release draft
0.2	Mar. 28, 2017	Peifu Jiang	Update Chapter 3.1 key encryption flow Update Chapter 4.1 key decryption flow Add Chapter 3.2 key wrapper tool usage Add Chapter 4.2 key provision tool usage
0.3	Apr.26, 2017	Peifu Jiang	Update Chapter 4.1 key type definition Update Chapter 5 Provision API
0.4	May.03, 2017	Peifu Jiang	Update Chapter 3.2 Key Wrapper Tool Usage Update Chapter 4.2 Key Decryption Flow Update Chapter 4.3.Key Provisioning Flow
0.5	Apr.20, 2018	Peifu Jiang	Add HDCP RX14 key and Keymaster attestation key Add API to get key checksum Add API to format RPMB Add name argument in store/query API
1.0	Feb.14, 2020	Peifu Jiang	Update version for provision 1.0 release
2.0	Apr.15, 2020	Liqiang Jin	Add HDCP RX22 key Add Nagra UUID and SECRET key Add Provision PFID and PFPK key Update provision flow Update version for provison 2.0 release
2.1	Apr.21, 2020	Liqiang Jin	Update provision_keywrapper tool usage information
2.2	Apr.22, 2020	Peifu Jiang	Add provision overview Update provision use case
2.3	Apr.26, 2020	Peifu Jiang	Update Chapter 3.3 to correct key type in Factory User mode Update Chapter 4.4 to highlight PFID and PFPK provisioning
2.4	Aug.08, 2020	Peifu Jiang	Update Chapter 2.2 to add new key types: NETFLIX_MGKID ATTEST_DEV_ID and WIDEVINE_CAS Update Chapter 3 and 5 to add key delete API Update PCPK description for chipset S905X4/S905C2
2.5	Mar.10,2021	Liqiang Jin	Add chipset family S905Y4/A311D2 support
2.5.1	Jun.24, 2021	Liqiang.jin	Add chipset family T982/T965D4/S905C3 support Add new key type: CIPLUS_ECP, DOLBY_ID
2.6	Jul.12, 2021	Liqiang Jin	Delete Chapter 2.4(Provision Overview) Delete Figuer 3-1(Secure Key Encryption Flow)
3.0	Oct.30, 2021	Liqiang Jin	Update key tool usage for S905X4 or later SoCs
3.1	Jan.10, 2022	Liqiang Jin	Update DGPK key tool usage
3.2	Feb.25, 2022	Liqiang Jin	Update document name and font
3.3	Apr.03, 2022	Peifu Jiang Liqiang Jin	Add more guidelines for field provisioning Add new DAC and Keybox Sha256 API

1. Preface

This document is subject to continuous developments and improvements. The information contained in this document is given by Amlogic in good faith. Amlogic shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use thereof.

1.1. Document Confidentiality

This document is confidential. This document may only be distributed by Amlogic to parties that have signed the Amlogic Software License Agreement (SLA).

1.2. Disclaimer

Amlogic reserves the right to modify and revise the specification and technology described here. All Amlogic technology is subject to contract.

2. Introduction

This document is intended primarily for the customers or partners of Amlogic. It provides an overview of the high level design of the secure key provisioning process, as well as how to provision a DRM key or certificate on Amlogic platform, how to use the provision PC tool to encrypt secure keys and how to decrypt the keys by provision application.

2.1. Definitions and Acronyms

- AES: Advanced Encryption Standard
- CA: Client Application
- DAC: Device Authentication Code
- DGPK: Device Global Purpose Key
- EPEK: Encrypted PEK
- GCM: Galois Counter Mode
- HMAC: Hash-based Message Authentication Code
- IPC: Inter Process Communication
- PEK: Provision Encryption Key
- PPK: Provision Protect Key
- PCPK: Provision Common Protect Key
- PFPK: Provision Field Protect Key
- PFID: Provision Field ID
- REE: Rich Execution Environment
- TA: Trusted Application
- TEE: Trusted Execution Environment

2.2. Scope

This document is mainly appropriate for the following Chipset Family:

- SC2: S905X4 S905C2 S905C2L

2.3. Secure Key Type

Secure key includes DRM key, HDCP key, Keymaster key and so on.

1) DRM Key:

- Widevine
  Provider: Google
  Key name: widevinekeybox.bin
- PlayReady
  Provider: Microsoft
  Key name: bgroupcert.dat zgpriv_protected.dat

2) HDCP Key:

- HDCP TX1.4 key
  Provider: DCP LLC
  Key name: hdcp_tx14.bin
- HDCP TX2.2 key
  Provider: DCP LLC
  Key name: hdcp_tx22.bin
- HDCP RX1.4 key
  Provider: DCP LLC
  Key name: hdcp_rx14.bin

- HDCP RX2.2 key
  Provider: DCP LLC
  Key name: hdcp_rx22.bin
- HDCP RX2.2 FW key
  Provider: DCP LLC
  Key name: hdcp_rx22_fw.bin
- HDCP RX2.2 FW PRIVATE key
  Provider: DCP LLC
  Key name: hdcp_rx22_fw_private.bin

3) Keymaster Key:

- Attestation key
  Provider: Google
  Key name: attestationkeybox.bin

Table 2-1 shows all supported secure key types and definitions.

Key ID	Key Type	Comments
0x00000011	KEY_TYPE_WIDEVINE	
0x00000021	KEY_TYPE_PLAYREADY_PRIVATE	
0x00000022	KEY_TYPE_PLAYREADY_PUBLIC	
0x00000031	KEY_TYPE_HDCP_TX14	
0x00000032	KEY_TYPE_HDCP_TX22	
0x00000033	KEY_TYPE_HDCP_RX14	
0x00000034	KEY_TYPE_HDCP_RX22_WFD	
0x00000035	KEY_TYPE_HDCP_RX22_FW	
0x00000036	KEY_TYPE_HDCP_RX22_FW_PRIVATE	
0x00000041	KEY_TYPE_KEYMASTER	Only for Android 8 and below version
0x00000042	KEY_TYPE_KEYMASTER3	Keymaster Attestation key
0x00000043	KEY_TYPE_KEYMASTER3_ATTEST_DEV_ID_BOX	Keymaster Attestation Device ID Box (optional)
0x00000051	KEY_TYPE_EFUSE	
0x00000061	KEY_TYPE_CIPLUS	
0x00000062	KEY_TYPE_CIPLUS_ECP	
0x00000071	KEY_TYPE_NAGRA_DEV_UUID	
0x00000072	KEY_TYPE_NAGRA_DEV_SECRET	
0x00000081	KEY_TYPE_PROVISION_PFID	Used for upgrading keybox by OTA
0x00000082	KEY_TYPE_PROVISION_PFPK	Used for upgrading keybox by OTA
0x00000091	KEY_TYPE_YOUTUBE_SECRET	
0x000000a2	KEY_TYPE_NETFLIX_MGKID	
0x000000b1	KEY_TYPE_WIDEVINE_CAS_KEY	
0x000000c1	KEY_TYPE_DOLBY_ID	
0xFFFFFFFF	KEY_TYPE_INVALID	

Table 2-1 Secure Key Type Definition

2.4. Provision Components

Secure keys can be provisioned on devices by TEE key provision tools. TEE key provision tools consist of the following components:

- **DGPK Deriving Script**
  dgpk1_derive.py
  A python script which can be used to generate DGPK1, and DGPK1 can be used to generate PCPK.
  dgpk2_derive.py

A python script which can be used to generate DGPK2, and DGPK2 can be used to generate PFPK.

- **Efuse Pattern Deriving Script**
  dgpk1_efuse_derive.py
  A python script which can be used to generate DGPK1 Efuse Pattern.
  dgpk2_efuse_derive.py
  A python script which can be used to generate DGPK2 Efuse Pattern.
- **PCPK Deriving Script**
  pcpk_derive.py
  A python script which can be used to generate PCPK.
- **DAC Deriving Script**
  dac_derive.py
  A python script which can be used to derive DAC from PFPK and PFID.
  The derived DAC can be used for Server Communication Authentication in field provisioning.
- **PEK Deriving Tool**
  pek_derive.py
  A python script which can be used to generate PEK.
- **Key wrap PC tool**
  provision_keywrapper for Linux
  windows_provision_keywrapper.exe for Windows
  A PC tool which can encrypt keybox using AES-GCM-128 algorithm.
- **Key provision CA tool**
  tee_provision
  A Client Application (CA) which can store encrypted keybox into secure storage on devices.
  It will be automatically installed on device.
- **Key provision CA library**
  libprovision.so
  The provision library is used by provision CA to communicate to TA . The specific provision APIs are defined in the header file. The CA library and header files can be used to integrate the provisioning functions in user applications.
  It will be automatically installed on device.
- **Key provision TA binary**
  e92a43ab-b4c8-4450-aa12b1516259613b.ta
  A Trusted Application (TA) which can decrypt the keybox using AES-GCM-128 decryption and store the keybox in secure storage.
  It will be automatically installed on device.

The above components and tools are located in:

- *${ANDROID_SDK}/vendor/amlogic/common/provision*
- *${BUILDROOT_SDK}/vendor/amlogic/provision*

3. Provision Tool Usage

3.1. DGPK1 Key Tool

DGPK1 key tool is used to generate DGPK1. DGPK is Device General Purpose Key which is stored in OTP. DGPK1 is a model unique key. PCPK is derived from DGPK1. PCPK is also a model unique key.

- **dgpk1_derive.py**

DGPK1 can be generated by ODMs themselves. It can also be generated by the following key tool.
Key tool usage:

```
$cd vendor/amlogic/common/provision/tool/sc2
$./sc2_dgpk1_derive.py
sc2-dgpk1/sc2_dgpk1.bin derived successfully
```

- **dgpk1_efuse_derive.py**

This tool is used to generate DGPK1 Efuse pattern.
Key tool usage:

```
$cd vendor/amlogic/common/provision/tool/sc2
$./sc2_dgpk1_efuse_derive.py --dgpk1 sc2-dgpk1/sc2_dgpk1.bin
sc2-dgpk1-efuse-pattern/sc2_dgpk1_efuse_pattern.bin.efuse derived successfully
```

- **pcpk_derive.py**

The PCPK is derived from DGPK1. The derived PCPK is used to encrypt the keybox.
Key tool usage:

```
$cd vendor/amlogic/common/provision/tool/sc2
$./sc2_pcpk_derive.py --dgpk1 sc2-dgpk1/sc2_dgpk1.bin
sc2-pcpk/sc2_pcpk.bin derived successfully
```

3.2. DGPK2 Key Tool

DGPK2 key tool is used to generate DGPK2. DGPK2 is also stored in OTP. DGPK2 is a device unique key. PFPK is derived from DGPK2. PFPK is also a device unique key.

- **dgpk2_derive.py**

DGPK2 can be generated by ODMs themselves. It can also be generated by the following key tool.
Key tool usage:

```
$cd vendor/amlogic/common/provision/tool/sc2
$./sc2_dgpk2_derive.py --count 8
sc2-dgpk2/sc2_dgpk2_000000001.bin derived successfully
sc2-dgpk2/sc2_dgpk2_000000002.bin derived successfully
sc2-dgpk2/sc2_dgpk2_000000003.bin derived successfully
sc2-dgpk2/sc2_dgpk2_000000004.bin derived successfully
sc2-dgpk2/sc2_dgpk2_000000005.bin derived successfully
sc2-dgpk2/sc2_dgpk2_000000006.bin derived successfully
sc2-dgpk2/sc2_dgpk2_000000007.bin derived successfully
sc2-dgpk2/sc2_dgpk2_000000008.bin derived successfully
8 dgpk2 files derived successfully
```

- **pfid_derive.py**

This tool is used to generate PFID. PFID is a device unique key.
Key tool usage:

```
$cd vendor/amlogic/common/provision/tool/sc2
```

```
$./sc2_pfid_derive.py --count 8
sc2-pfid/sc2_pfid_000000001.bin derived successfully
sc2-pfid/sc2_pfid_000000002.bin derived successfully
sc2-pfid/sc2_pfid_000000003.bin derived successfully
sc2-pfid/sc2_pfid_000000004.bin derived successfully
sc2-pfid/sc2_pfid_000000005.bin derived successfully
sc2-pfid/sc2_pfid_000000006.bin derived successfully
sc2-pfid/sc2_pfid_000000007.bin derived successfully
sc2-pfid/sc2_pfid_000000008.bin derived successfully
8 pfid files derived successfully
```

- **dgpk2_efuse_derive.py**

This tool is used to generate DGPK2 Efuse pattern. PFID is also included in this Efuse pattern.
Key tool usage:

```
$cd vendor/amlogic/common/provision/tool/sc2
$./sc2_dgpk2_efuse_derive.py --dgpk2 sc2-dgpk2/ --pfid sc2-pfid/
sc2-dgpk2-efuse-pattern/sc2_dgpk2_efuse_pattern_000000001.bin.efuse derived successfully
sc2-dgpk2-efuse-pattern/sc2_dgpk2_efuse_pattern_000000002.bin.efuse derived successfully
sc2-dgpk2-efuse-pattern/sc2_dgpk2_efuse_pattern_000000003.bin.efuse derived successfully
sc2-dgpk2-efuse-pattern/sc2_dgpk2_efuse_pattern_000000004.bin.efuse derived successfully
sc2-dgpk2-efuse-pattern/sc2_dgpk2_efuse_pattern_000000005.bin.efuse derived successfully
sc2-dgpk2-efuse-pattern/sc2_dgpk2_efuse_pattern_000000006.bin.efuse derived successfully
sc2-dgpk2-efuse-pattern/sc2_dgpk2_efuse_pattern_000000007.bin.efuse derived successfully
sc2-dgpk2-efuse-pattern/sc2_dgpk2_efuse_pattern_000000008.bin.efuse derived successfully
8 dgpk2_efuse_pattern files derived successfully
```

- **pfpk_derive.py**

This tool is used to derive PFPK from DGPK2 and PFID. PFPK is a device unique key.
Key tool usage:

```
$cd vendor/amlogic/common/provision/tool/sc2
$./sc2_pfpk_derive.py --dgpk2 sc2-dgpk2/ --pfid sc2-pfid/
sc2-pfpk/sc2_pfpk_000000001.bin derived successfully
sc2-pfpk/sc2_pfpk_000000002.bin derived successfully
sc2-pfpk/sc2_pfpk_000000003.bin derived successfully
sc2-pfpk/sc2_pfpk_000000004.bin derived successfully
sc2-pfpk/sc2_pfpk_000000005.bin derived successfully
sc2-pfpk/sc2_pfpk_000000006.bin derived successfully
sc2-pfpk/sc2_pfpk_000000007.bin derived successfully
sc2-pfpk/sc2_pfpk_000000008.bin derived successfully
8 pfpk files derived successfully
```

- **dac_derive.py**

This tool can derive DAC from DGPK2 and PFID. The derived DAC can be used for Server Communication Authentication in field provisioning.
Key tool usage:

```
$cd vendor/amlogic/common/provision/tool/sc2
$./sc2_dac_derive.py --dgpk2 sc2-dgpk2/ --pfid sc2-pfid/
sc2-dac/sc2_dac_000000001.bin derived successfully
sc2-dac/sc2_dac_000000002.bin derived successfully
sc2-dac/sc2_dac_000000003.bin derived successfully
sc2-dac/sc2_dac_000000004.bin derived successfully
sc2-dac/sc2_dac_000000005.bin derived successfully
sc2-dac/sc2_dac_000000006.bin derived successfully
```

```
sc2-dac/sc2_dac_000000007.bin derived successfully
sc2-dac/sc2_dac_000000008.bin derived successfully
8 dac files derived successfully
```

NOTE

- The PFPK/PFID/DAC are used for field provisioning by OTA.
- The PFPK/PFID/DAC are device-unique files. They need to be associated together for field provisioning. E.g. sc2-dgpk2/sc2_dgpk2_**000000001**.bin is associated with sc2-pfid/sc2_pfid_**000000001**.bin and sc2-dac/sc2_dac_**000000001**.bin.
- The sc2_dgpk2_efuse_pattern includes both **DGPK2** and **PFID**. E.g. sc2-dgpk2-efuse-pattern/sc2_dgpk2_efuse_pattern_**000000001**.bin.efuse will include sc2-pfid/sc2_pfid_**000000001**.bin and sc2-dgpk2/sc2_dgpk2_**000000001**.bin.

- **How to provision Efuse pattern:**

Efuse pattern can be provisioned by USB Burning Tool.
It can also be provisioned manually using command console in U-boot:

```
# mmcinfo; fatload mmc 0 1080000 sc2_dgpk_efuse_pattern.efuse
# efuse amlogic_set 1080000
```

3.3. PEK Key Tool

This is a python script which can be used to generate PEK.
Key tool usage:

```
$cd vendor/amlogic/common/provision/tool/sc2
$ ./sc2_pek_derive.py --count 1024
1024 pek files derived successfully
```

3.4. Key Wrapper Tool

Key encryption tool is developed based on OpenSSL 1.0.1. It can be used to encrypt secure keys.
Tool usage:

```
$ ./provision_keywrapper -h
Amlogic Provision Keywrapper Tool v3.1.0
Usage: provision_keywrapper [-t <key type>] [-i <key file>] [-u <ta uuid>] [-e <pek file>] [-p <ppk file>]
[-m <mode>] [-a <key dir>] [-l] [-v] [-h]

  -t, --type        Input key type
  -i, --in          Input key or keybox file
  -u, --uuid        TA UUID which is optional
  -e, --pek         Provision Encryption Key File which is optional
  -p, --ppk         Provision Protect Key File
  -m, --mode        Encrypt mode(0: factory; 1: factory user; 2: field)
  -a, --all         Input key files directory
  -l, --list        List all key types
  -v, --version     Show tool version
  -h, --help        Show this help
```

- **-m**, this argument is used to indicate keybox mode. 0 is provision mode for bootloader stage factory provisioning. Typically USB burning tool is used for this provision mode. 1 is provision mode for user space factory provisioning. Some provision application or APK uses this provision mode. 2 is provision mode for OTA or other in-field provisioning.
- **-p**, this argument is used to indicate PPK. In factory or factory-user mode, PPK is PCPK. In field mode, PPK is PFPK.

- **-u,** this argument is used for extended secure key type.

The following commands show how to generate Widevine keybox in different modes:

- **Keybox for Factory mode.** Keybox is provisioned on device by USB_Burning Tool in factory production stage.

```
$./provision_keywrapper -t 0x11 -i widevinekeybox.bin -p sc2-pcpk/sc2_pcpk.bin
[PROVISION-KEYWRAPPER] Input:
[PROVISION-KEYWRAPPER]      Key File: widevinekeybox.bin
[PROVISION-KEYWRAPPER]      Provision Protect Key File: sc2-pcpk/sc2_pcpk.bin
[PROVISION-KEYWRAPPER]      Key Type: KEY_WIDEVINE(0x11)
[PROVISION-KEYWRAPPER]      Provision Encryption Key File: sc2-pek/sc2_pek_000000001.bin
[PROVISION-KEYWRAPPER] Output:
[PROVISION-KEYWRAPPER]      Output File: widevinekeybox.bin.factory.enc
```

- **Keybox for Factory User mode.** Keybox is provisioned on device by tee_provision tool, Provision apk or other user application in factory production stage.

```
$ ./provision_keywrapper -t 0x11 -i widevinekeybox.bin -p sc2-pcpk/sc2_pcpk.bin –m 1
[PROVISION-KEYWRAPPER] Input:
[PROVISION-KEYWRAPPER]      Key File: widevinekeybox.bin
[PROVISION-KEYWRAPPER]      Provision Protect Key File: sc2-pcpk/sc2_pcpk.bin
[PROVISION-KEYWRAPPER]      Key Type: KEY_WIDEVINE(0x11)
[PROVISION-KEYWRAPPER]      Provision Encryption Key File: sc2-pek/sc2_pek_000000001.bin
[PROVISION-KEYWRAPPER] Output:
[PROVISION-KEYWRAPPER]      Output File: widevinekeybox.bin.factory-user.enc
```

- **Keybox for Field mode.** Keybox is updated on device by OTA update application in field. Usually this mode is only used to update keybox by OTA.

```
$ ./provision_keywrapper -t 0x11 -i widevinekeybox.bin -p sc2-pfpk/sc2_pfpk_000000001.bin –m 2
[PROVISION-KEYWRAPPER] Input:
[PROVISION-KEYWRAPPER]      Key File: widevinekeybox.bin
[PROVISION-KEYWRAPPER]      Provision Protect Key File: sc2-pfpk/sc2_pfpk_000000001.bin
[PROVISION-KEYWRAPPER]      Key Type: KEY_WIDEVINE(0x11)
[PROVISION-KEYWRAPPER]      Provision Encryption Key File: sc2-pek/sc2_pek_000000001.bin
[PROVISION-KEYWRAPPER] Output:
[PROVISION-KEYWRAPPER]      Output File: widevinekeybox.bin.field.enc
```

3.5. Provision CA Tool

Provision CA tool is a client application which runs on device to store secure keyboxes.

```
#tee_provision -h
Amlogic Provision Tool v3.3.0
Usage: tee_provision [-i <file>] [-n <name>] [-t <type>] [-e <nonce str>] [-u <ta uuid>] [-q] [-l] [-c] [-p]
[-a] [-v] [-h]

    -i, --in            Input data file
    -n, --name          Specify key name for keymaster/ciplus
    -t, --type          Specify key type
    -e, --nonce         nonce string for calculating dac
    -q, --query         Query key provisioned or not
    -d, --delete        Delete provisioned key
    -l, --list          List all key types
    -c, --checksum      Show key checksum
    -p, --pfid          Get provision field id
```

```
        -a, --dac               Get device authentication code
        -u, --uuid              TA UUID
        -s, --sha256            calc raw key sha256
        -v, --version           Show tool version
        -h, --help              Show this help
```

Provision Widevine keybox in Factory User mode:

```
    $ ./tee_provision -i widevinekeybox.bin.factory-user.enc
```

Query Widevine keybox:

```
    $ ./tee_provision -q -t 0x11
```

4. Provision Use Case

There are 3 use cases for secure key provision: **Factory, Factory User, Field.**

- **Factory Mode**

In factory manufacturing line, provision keyboxes in bootloader stage.
USB burning tool can be used to burn keyboxes.

- **Factory User Mode**

In factory manufacturing line, provision keyboxes in user space.
Provision CA Tool or 3rd party developed provision application can be used to provision keyboxes.

- **Field Mode**

In field, provision keyboxes by OTA.
OTA Updating Tool or application can be used to update existing keyboxes or write new keyboxes.

4.1. Factory Mode

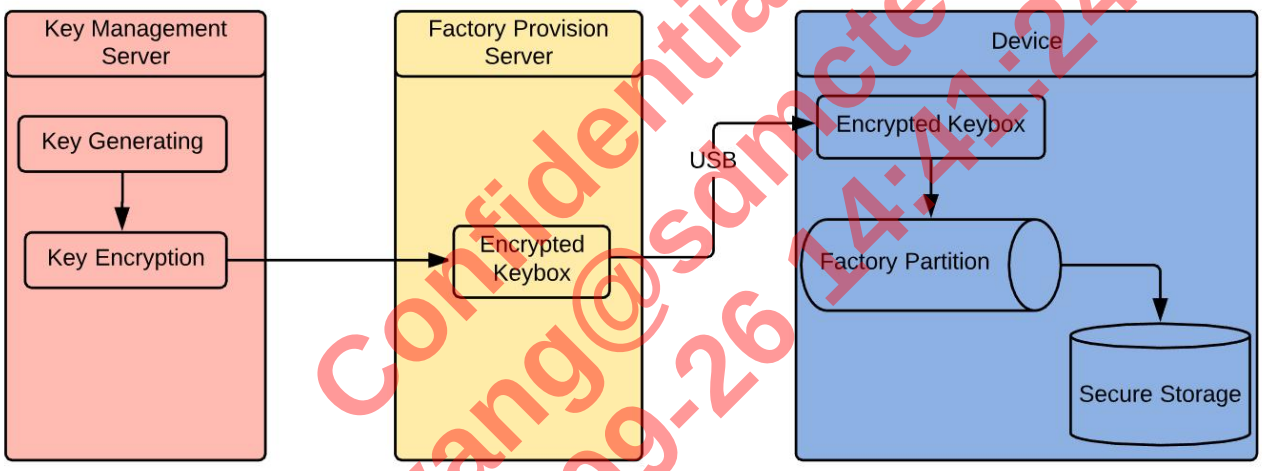

Figure 4-1 Secure Key Factory Provison Flow

Requirements:

PPK(PCPK), PEK

1) On PC server, use Tool to generate DGPK, DGPK Efuse Pattern, PCPK, PEK
2) On PC server, use Key Wrapper Tool to encrypt key data (use PCPK to do encryption)
3) On Device, write the DGPK Efuse Pattern into device Efuse
4) On Device, use USB Burning tool to burn keyboxes

Note

- For how to use USB Burning Tool, please refer to "USB Burning Tool User Guide".

4.2. Factory User Mode

Requirements:

PPK(PCPK), PEK

1) On PC server, use Tool to generate DGPK, DGPK Efuse Pattern, PCPK, PEK
2) On PC server, use Key Wrapper Tool to encrypt key data (use PCPK to do encryption)

3) On Device, write the DGPK Efuse Pattern into device Efuse
4) On Device, use provision CA tool or 3rd party provision application to provision keyboxes

Note

- Depending on the specific manufacturing production-line, ODMs can choose either Factory Mode or Factory User Mode to do factory provisioning.

4.3. Field Mode

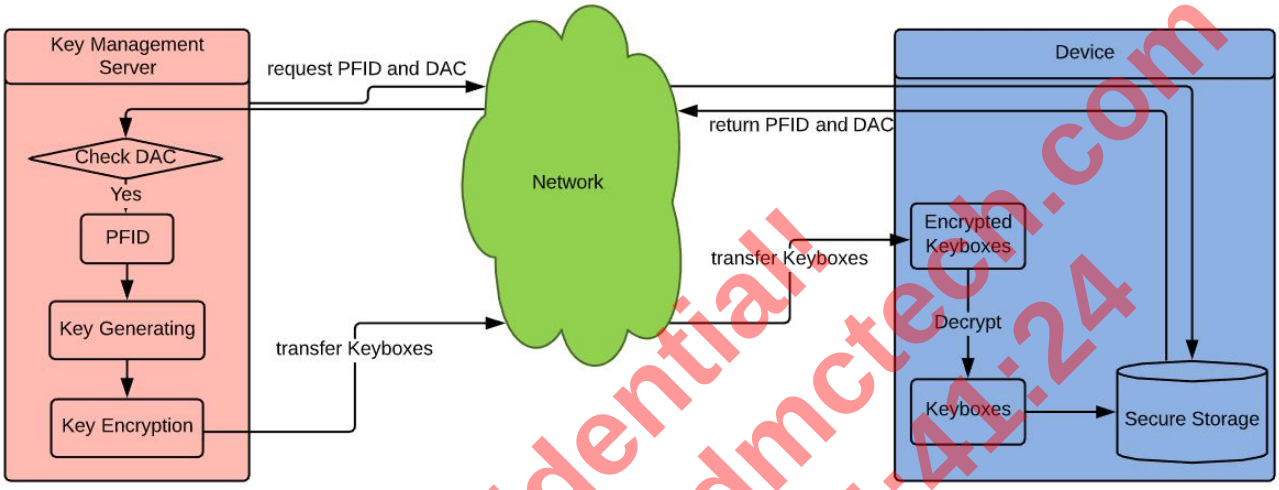

Figure 4-2 Secure Key Field Provision Flow

Requirements:

PPK(PFPK), PEK

Provision Flow:

1) On PC server, use Key Wrapper Tool to encrypt key data (use PFPK to do encryption)
2) On Device, OTA application gets PFID and DAC by invoking Provision APIs provided by provision library
3) On Device, OTA application sends PFID/DAC to OTA server to request device specific encrypted keyboxes.
4) On Device, OTA application invokes provision library to update or write keyboxes.

NOTE

The above Field Mode provision flow is recommended by Amlogic. ODMs can define their own field provision flows.

- PFID is per-device unique ID.
- PFPK is per-device AES key. PFPK is bound with Provision Field ID (PFID).
- To support Field Mode provisioning, **DGPK2** and **PFID MUST** be provisioned during device factory manufacturing stage.
- This provision flow can only be used for OTA or other field key provisioning processes.

Here are the steps on how to **enable field provisioning** on devices:

- STEP-1 Generate PFPK and PFID for devices.
  - STEP-1.1 Run pfid_derive.py to generate **PFID** for devices.
  - STEP-1.2 Run dgpk2_derive.py to generate **DGPK2** for devices.
  - STEP-1.3 Run pfpk_derive.py to generate **PFPK** for devices.
  - STEP-1.4 Run dac_derive.py to generate **DAC** files for devices.
  - STEP-1.5 Run dgpk2_efuse_derive.py to generate **DGPK2_FPID EFUSE** pattern for devices.
- STEP-2 Provision **DGPK2_FPID EFUSE** pattern on devices in **factory manufacturing line.**
  - STEP-2.1 Use USB Burning Tool or Uboot command line to provision **DGPK2_FPID EFUSE** pattern on the device.
- STEP-3 Deploy PFID/PFPK/DAC on Field Provision Server (Key Management Server in Figure 4-2).

  - STEP-3.1 Deploy **PFID** files generated in STEP-1.1 on Field Provision Server.
  - STEP-3.2 Deploy **PFPK** files generated in STEP-1.3 on Field Provision Server.
  - STEP-3.3 Deploy **DAC** files generated in STEP-1.4 on Field Provision Server.
- STEP-4 Deploy encrypted keyboxes on Field Provision Server
  - STEP-4.1 Run provision_keywrapper to generate **PFPK encrypted keyboxes**
  - STEP-4.2 Deploy the **PFPK encrypted keyboxes** on Field Provision Server.

In above STEP-1, the detailed usage of key tool can be found in Chapter 3.2 DGPK2 Key Tool. After STEP-2, the devices have the unique PFID programmed and can derive PFPK from DGPK2. STEP-3 and STEP-4 can ensure all required materials are deployed on Field Provision Server, and the server is ready to provide field provisioning service.

Here are the steps on how to **perform field provisioning** on devices:

- STEP-1 The device sends field provisioning request to Field Provision Server (Key Management Server in Figure 4-2).
  - STEP-1.1 The OTA application invokes **PFID Get API** (Chapter 5.5) to get **PFID**.
  - STEP-1.2 The OTA application invokes **DAC Get API** (Chapter 5.6) to get **DAC**.
  - STEP-1.3 The OTA application sends **PFID** and **DAC** to Field Provision Server.
- STEP-2 The server handles the device request.
  - STEP-2.1 The server checks database to ensure the device provided **PFID** is valid.
  - STEP-2.2 The server performs **DAC** validation.
  - STEP-2.3 The server returns specific encrypted keyboxes to the device.
- STEP-3 The device performs key provisioning.
  - STEP-3.1 The OTA application invokes **Key Store API** (Chapter 5.1) to write keyboxes.
  - STEP-3.2 The OTA application invokes **Key Query API** (Chapter 5.2) to check the keyboxes are written correctly.

In above steps, the detailed usage of provision API can be found in Chapter 5 Provision API.

5. Provision API

The API definitions can be found in the following header files:

- ${**ANDROID_SDK**}/vendor/amlogic/common/provision/ca/include/provision_api.h.
- ${**BUILDROOT_SDK**}/vendor/amlogic/provision/ca/include/provision_api.h.

5.1. Key Store API

Secure key store API can be used to provision key on device.

```
int32_t key_provision_store(
    [in] uint8_t *name_buff,
    [in] uint32_t name_size,
    [in] uint8_t *key_buff,
    [in] uint32_t key_size);
```

Description

The key_provision_store function writes key_size bytes from the buffer pointed by key_buff to the Secure Storage.

Parameters

- name_buff: The keybox name (deprecated, should be NULL)
- name_size: The keybox name size (deprecated, should be 0)
- key_buff: The keybox buffer
- key_size: The number of keybox bytes

Return Code

- 0x0: success
- ~0x0: Error code. Error codes are described in Table 5-1.

Error Code	Definition	Possible Causes
0xFFFF0006	TEE_ERROR_BAD_PARAMETERS	The CA and TA library versions do not match; The key size is incorrect;
0xFFFF0007	TEE_ERROR_BAD_STATE	DGPK is not burnt.
0xFFFF0008	TEE_ERROR_ITEM_NOT_FOUND	TA library does not exist in system.
0xFFFF3071	TEE_ERROR_MAC_INVALID	The PCPK which is used to encrypt keybox is incorrect.

Table 5-1 Provision Error Code

5.2. Key Query API

Secure key query API can be used to check whether the key is provisioned or not on device.

```
int32_t key_provision_query(
    [in] uint8_t *name_buff,
    [in] uint32_t name_size,
    [in] uint32_t key_type,
    [out] uint32_t *storage,
    [out] uint32_t *key_size);

int32_t key_provision_query_v2(
    [in] uint8_t *name_buff,
    [in] uint32_t name_size,
    [in] uint32_t key_type,
    [in] uint8_t *uuid,
    [out] uint32_t *storage,
    [out] uint32_t *key_size);
```

Parameters

- key_type: The keybox type
- name_buff: The keybox name (deprecated, should be NULL)
- name_size: The keybox name size (deprecated, should be 0)
- uuid: TA uuid of the keybox with extended key type
- storage: key storage location (0x80000000: TEE Partition; 0x80000100: RPMB)
- key_size: The number of keybox bytes

Return Code

- 0x0: success
- ~0x0: Error code. Error codes are described in Table 5-1.

5.3. Key Checksum API

Secure key checksum API can be used to get the checksum of the provisioned key on device.

```
int32_t key_provision_checksum(
    [in] uint32_t key_type,
    [in] uint8_t *name_buff,
    [in] uint32_t name_size,
    [out] uint8_t *checksum);

int32_t key_provision_checksum_v2(
    [in] uint32_t key_type,
    [in] uint8_t *name_buff,
    [in] uint32_t name_size,
    [in] uint8_t *uuid,
    [out] uint8_t *checksum);
```

Parameters

- key_type: The keybox type
- name_buff: The keybox name (deprecated, should be NULL)
- name_size: The keybox name size (deprecated, should be 0)
- uuid: TA UUID of the keybox with extended key type
- checksum: The checksum of keybox

Return Code

- 0x0: success
- ~0x0: Error. Error codes are described in Table 5-1

5.4. Key Delete API

Secure key delete API can be used to remove the provisioned key on device.

```
int32_t key_provision_delete(
    [in] uint32_t key_type
    [in] uint8_t *uuid);
```

Parameters

- key_type: The keybox type
- uuid: TA UUID of the keybox with extended key type

Return Code

- 0x0: success
- ~0x0: Error

5.5. PFID Get API

PFID Get API can be used to get PFID on device.

```
int32_t key_provision_get_pfid(
        [out] uint8_t *pfid,
        [out] uint32_t *id_size);
```

Description

The key_provision_get_pfid function gets PFID from device.

Parameters

- pfid: PFID buffer
- id_size: PFID size

Return Code

- 0x0: success
- ~0x0: Error code

5.6. DAC Get API

DAC Get API can be used to get DAC on device.

```
int32_t key_provision_get_dac(
        [out] uint8_t *dac,
        [out] uint32_t *dac_size);

int32_t key_provision_get_dac_v2(
        [in] const uint8_t *nonce,
        [in] uint32_t nonce_size,
        [out] uint8_t *dac,
        [out] uint32_t *dac_size);
```

Description

The key_provision_get_dac function gets DAC from device.

Parameters

- dac: DAC buffer
- dac_size: DAC size
- nonce: the nonce string used to generate the DAC
- nonce_size: size of the nonce string used to generate the DAC

Return Code

- 0x0: success
- ~0x0: Error code

5.7. Keybox Sha256 Calc API

Keybox Sha256 Calc API can be used to calculate sha256 of raw key from encrypted keybox on device.

```
int32_t key_provision_calc_checksum(
        [in] const uint8_t *keybox,
        [in] uint32_t keybox_size,
        [out] uint8_t checksum[PROVISION_KEY_CHECKSUM_LENGTH]);
```

Description

Keybox Sha256 Calc API can be used to calculate sha256 of raw key from encrypted keybox on device.

Parameters

- keybox: encrypted key data
- keybox_size: encrypted key data size
- checksum: raw key data's sha256

Return Code

- 0x0: success
- ~0x0: Error code